

SIH26171

On-device Visual Perception for Light-weight Browser Agents

How Redact Agent maps to the problem statement, and what actually happens inside the agent when the user gives it a task.

Project: Redact Agent — Chrome MV3 extension

Team submission for: Smart India Hackathon 2025

Document scope: problem statement (verbatim), architecture mapping, step-by-step task execution.

1. The Problem Statement (SIH26171)

Title: On-device Visual Perception for Light-weight Browser Agents

1.1 Background

Agentic AI is becoming pervasive in day-to-day digital work. If such an agent had access to the user's **visual context and screen state**, it could assist in complex workflows and automate many tasks. Most agentic pipelines run **server-side**, which limits the kind of data users are willing to share. Running the agent **locally in the browser** removes that limitation — the machine handles perception, and only non-sensitive structure (screen layout, application fields, etc.) is sent to the server.

Modern browser primitives — **WebGPU**, **WebAssembly**, **ONNX Runtime Web** and **Transformers.js** — now make it practical to run lightweight ML models directly on the client. The aim is to bridge two environments: **server-side reasoning power** combined with **strict client-side data privacy**.

1.2 Description

Participants must build a **privacy-preserving vision agent that runs in the browser**. A local Vision Transformer (ViT) or equivalent computer-vision model reads the user's screen and takes decisions from it. If visual context must be sent to a server, the client **sanitizes sensitive / PII data** — using DOM tags, bounding-box redaction, blurring, or masking — *before* any network request is made.

Only anonymized, unidentifiable data is transmitted. The server is aware of this redaction scheme and processes the sanitized context accordingly, returning either processed data or an actionable command (e.g. *"click the submit button"*, *"scroll down"*) for the client to execute. Participants must balance the trade-off between **inference latency and accuracy**.

1.3 Expected Solution

A working prototype with the following components.

Client-side (extension / JS) running in a modern browser:

- **Local Vision Processing** — a client-side vision model (e.g. via WebGPU) that evaluates the current screen state.
- **Privacy-Preserving Filter** — a mechanism for sanitizing sensitive/personal visual data through local bounding-box redaction, semantic obfuscation, masking, etc. This must be clearly demonstrated.

Server-side:

- **Integration** — transmits the anonymized visual context to a central LLM/VLM, which interprets the sanitized data and returns either processed data (for the client to ingest) or a UI action for the client to execute.
- Participants may use any offline-deployable open-source / open-weights model; cloud-hosted versions are allowed during SIH.
- An **end-to-end task** that assists the user must be demonstrated.

1.4 Evaluation Metrics

#	Weight	Metric
1	25%	Accuracy of visual context extracted from the screen
2	20%	Recall and precision for detection of sensitive / PII data
3	20%	Precision of redaction
4	20%	Client-side resource utilization
5	15%	Overall end-to-end latency of the provided task

2. How Redact Agent Solves It

The submission is a Chrome Manifest V3 extension that lives in the side panel. Four cooperating layers implement the problem statement's split of *local perception*, *local redaction*, and *server reasoning over anonymized data*.

Layer	Purpose	Where it runs
Local perception	Reads the current page: DOM scan, screenshot capture, in-browser OCR (Tesseract.js WASM), face + PII vision (YuNet ONNX + OpenCV in Web Workers).	100% in-browser
Local privacy engine	Pattern + context analysis, DOM / OCR / vision fusion (IoU dedup + stable IDs), opaque-block redaction on an OffscreenCanvas, and a verification re-scan.	100% in-browser
Remote reasoning (LLM)	Gemini (or a local VLM via the optional FastAPI server). Sees only detection IDs, category labels, confidence scores and safe page metadata — never raw PII, never raw pixels.	Cloud, or the local server
Executor	Runs the LLM's tool calls in the browser after every call passes the ToolValidator allowlist and typed-parameter checks.	In-browser (extension context)

2.1 Data-flow guarantee

The LLM never sees:

- raw text values matched by the detectors
- bounding-box coordinates
- DOM element selectors
- raw screenshot pixels

The LLM only ever sees:

- the user's task text
- stable detection IDs of the form `det_0`, `det_1`, ...
- category labels such as `email`, `phone`, `face`
- confidence scores and per-category count summaries
- safe page context (host, title, viewport size)

This is exactly the split the problem statement asks for: perception and redaction happen on the client; the server only ever receives anonymized structural context and replies with actionable commands.

3. End-to-End: What Happens When You Give the Agent a Task

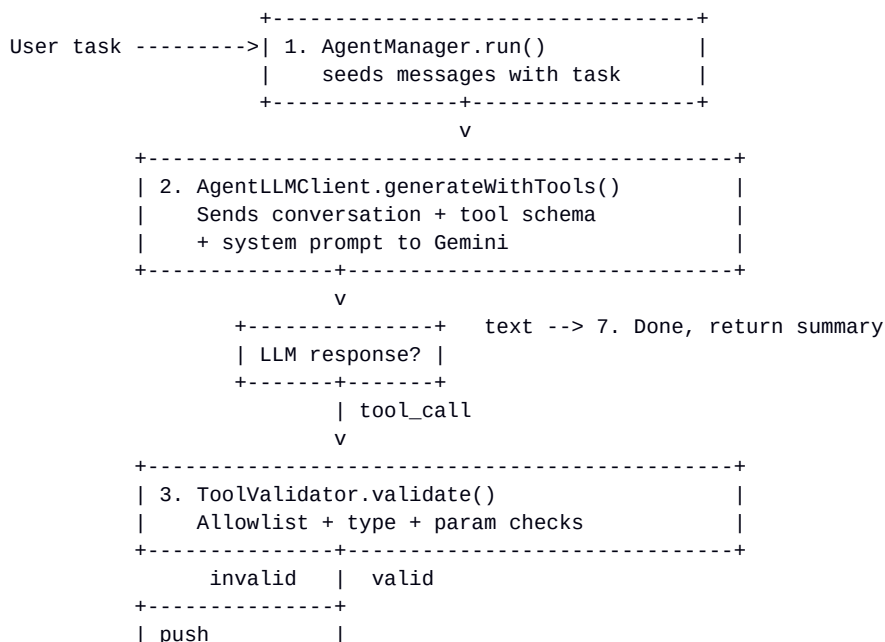
The loop lives in `src/agent/agentManager.js` and follows a classical **Reason** → **Act** → **Observe** cycle, capped at `MAX_AGENT_STEPS = 15` with a same-tool-in-a-row guard of 2.

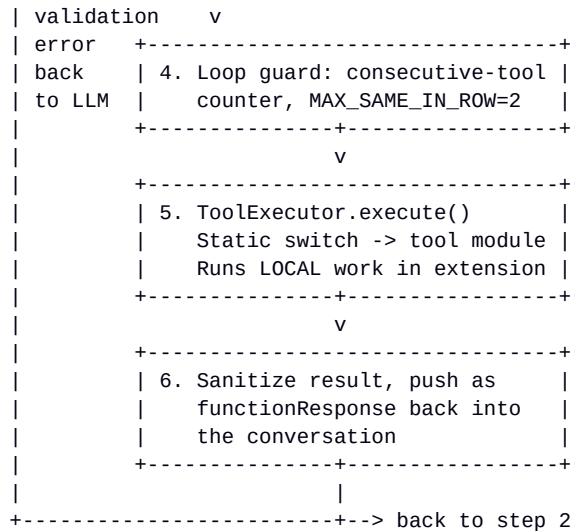
3.1 The tools the LLM may call

The LLM cannot inject code. It can only pick a tool name from the fixed list below and supply parameters that match the schema. Every tool executes **inside the extension**.

Tool	What it does (locally)	What is sent back to the LLM
<code>get_page_context</code>	Read host, title, viewport of the active tab.	Safe metadata only.
<code>scan_dom</code>	Pattern + context PII detection over DOM text.	Detection count and category summary.
<code>take_screenshot</code>	Capture the active tab into local memory.	Only the image dimensions.
<code>scan_ocr</code>	Run Tesseract.js WASM over the screenshot.	Word count and detection count.
<code>fuse_detections</code>	Merge DOM + OCR + vision results, assign stable IDs, run YuNet face detection.	List of <code>det_N</code> IDs + categories and confidences.
<code>redact</code>	Paint opaque blocks over regions on an <code>OffscreenCanvas</code> .	Count of regions masked.
<code>verify_redaction</code>	Re-scan the redacted image to confirm nothing sensitive remains.	Boolean and remaining count.
<code>click_element / navigate_to / open_tab</code>	Browser actions in the active tab.	Success or failure only.

3.2 The loop, step by step





Each turn does the following.

- **Reason** — the LLM sees the running transcript and picks the next tool.
- **Validate** — ToolValidator rejects unknown tools, wrong types, malformed IDs (must match det_N), disallowed redaction methods, non-http URLs, and oversized strings.
- **Guard** — a counter blocks the same tool from firing three or more times in a row without progress.
- **Act** — ToolExecutor routes via a hardcoded switch (no dynamic dispatch, no eval) to the matching module under src/agent/tools/.
- **Observe** — the tool returns a **sanitized** result: counts, categories, IDs. The result is added to the conversation as a functionResponse.
- **Stop** — when the LLM returns a plain text message (no tool call), that is the final summary. Otherwise the loop caps at 15 steps.

3.3 A concrete walk-through

User task: "Redact all personal information on this page."

#	LLM decides	What runs locally	What the LLM is told back
1	scan_dom	Pattern + context analyzers walk the DOM text; regex and heuristics assign confidences.	{ detectionCount: 12, categorySummary: { email: 4, phone: 3, name: 5 } }
2	take_screenshot	chrome.tabs.captureVisibleTab → stored in AgentState — pixels never leave the extension.	{ width: 1440, height: 900 }
3	scan_ocr	Tesseract.js WASM runs on the screenshot; extracted words pass through the same pattern + context analyzers.	{ ocrWordCount: 318, detectionCount: 4, categorySummary: { email:1, phone:1, cardNumber:2 } }
4	fuse_detections	IoU-based dedup across DOM + OCR + YuNet face detections (ONNX Runtime Web worker); each surviving detection is assigned a stable det_N ID.	{ totalDetections: 14, ids: [det_0, ..., det_13], visionEnabled: true }
5	redact (ids: all 14, method: opaque)	ScreenshotRedactor paints black rectangles onto an OffscreenCanvas for each detection's bbox; the result becomes the redacted screenshot.	{ redactedCount: 14 }
6	verify_redaction	Re-scan the redacted image to confirm nothing sensitive is still visible.	{ verified: true, remainingCount: 0 }
7	text response	—	"Done — 14 items redacted: 5 names, 5 emails, 3 phone numbers, 2 card numbers, 1 face. No sensitive data remains visible."

The user watches this Observe → Decide → Act log fill in the side panel and sees the redacted screenshot when the task ends.

User task (browser action): "Open the Repositories tab on this GitHub profile."

#	LLM decides	What runs locally	LLM sees back
1	click_element(text: "Repositories")	Content script finds a visible element by accessible label and clicks it.	{ success: true, matchedBy: 'text' }
2	text response	—	"Clicked the Repositories tab."

If step 1 had returned NOT_FOUND, the system prompt tells the LLM to fall back to get_page_context → navigate_to(url).

3.4 Where the SIH26171 requirements are met in code

Requirement	Where it lives in the repo
Local vision processing in-browser (WebGPU / WASM)	src/vision/yunet.worker.ts src/vision/opencv.worker.ts src/privacy/ocrAnalyzer.js (Tesseract.js WASM)
Privacy-preserving filter (bounding-box redaction on the client)	src/privacy/screenshotRedactor.js src/privacy/detectionFusion.js
Only anonymized context reaches the server	src/agent/llmClient.js — sanitized functionResponse payloads built in src/agent/agentManager.js
Server returns actionable commands the client executes	Tool-calling loop in src/agent/agentManager.js Optional local VLM backend in server/
End-to-end user-assist demo	Quick-task buttons in src/components/AgentMode.jsx Demo page: test/pii-test-page.html

3.5 How the design targets each evaluation metric

Metric	Design choice
Visual context accuracy (25%)	Multi-signal fusion: DOM parse + OCR + YuNet vision. Gives coverage across text, canvas, and image regions instead of relying on OCR alone.
PII detection recall & precision (20%)	Regex patterns and context analysis (proximity to labels like <i>email:</i> , <i>phone</i>); NER-style scoring in contextAnalyzer.js reduces false positives.
Redaction precision (20%)	Every detection carries a bounding box from its source (DOM rect or OCR word geometry); screenshotRedactor paints an opaque block per box on an OffscreenCanvas, and verify_redaction re-scans to prove nothing leaked through.
Client-side resource utilization (20%)	Vision and OCR run in dedicated Web Workers (yunet.worker.ts, opencv.worker.ts) to keep the side panel responsive; models load lazily on first use.
End-to-end latency (15%)	Loop hard-caps at 15 steps with a same-tool guard; low temperature: 0.1 reduces reasoning drift; only summary metadata (not pixels) is sent per turn.

4. Quick Reference

- **Entry point of the loop:** `AgentManager.run()` called from `src/components/AgentMode.jsx`.
- **Security boundary:** every LLM tool call → `ToolValidator.validate()` → allowlist + typed parameter check.
- **Sanitization boundary:** tool results → `AgentLLMClient.buildFunctionResponseMessage()` → only IDs, counts, and category labels reach the next LLM turn.
- **Full architecture diagram:** `docs/ARCHITECTURE.md`.